LITHO MAINNET 9005 · EXCHANGE INTEGRATION

LITHO Mainnet Exchange Integration

Native LITHO · LEP100 · Node · API · Questionnaire

Version: Technical draft · 2026-07-29

Network: `lithosphere_9005-1` / EVM `9005`

This package contains the master exchange questionnaire response, node installation guide, API reference, LEP100 integration guide, and external release checklist.

> **Release gate:** Do not submit as final until the production explorer, public release artifacts, mainnet LEP100 registry, wallet support, confirmation policy, and transfer-opening time are approved.

LITHO Mainnet Exchange Integration Package

Last reviewed: 2026-07-29

Document status: **technical draft complete; external submission gated**. The base chain and node APIs are live. The production explorer, public binary release URL, public snapshot URL, approved mainnet LEP100 registry, formal deposit-confirmation policy, and confirmed mainnet wallet support must be closed before this package is represented as a final exchange listing pack.

This is the master response for exchanges integrating the native LITHO coin and LEP100 fungible tokens. Detailed implementation guidance is provided in:

- Exchange node installation
- Exchange API reference
- LEP100 integration
- External release checklist

Exchange quick reference

Field	Value
Network	Lithosphere Mainnet
Native coin name	Lithosphere
Native coin symbol	`LITHO`
Base denomination	`ulitho`
Decimal precision	18; `1 LITHO = 10^18 ulitho`
Cosmos chain ID	`lithosphere_9005-1`
EVM chain ID	`9005` (`0x232d`)
Consensus	CometBFT proof of stake; not proof of work
Account model	Account-based; not UTXO
Genesis SHA-256	`13e4875b4a9dddc63bdfbd4968c7265f9bbc49218b59c5b49231a56fa313046f`
Approved binary SHA-256	`0546677a9cf3a7f458797b65181a46f21c89185933e832d89ce728a144fd258c`
Genesis URL	`https://rpc-mainnet.litho.ai/genesis.json`
EVM JSON-RPC	`https://rpc-mainnet.litho.ai`
EVM WebSocket	`wss://rpc-mainnet.litho.ai/websocket`
Cosmos REST	`https://api-mainnet.litho.ai`
gRPC	`grpc-mainnet.litho.ai:9090`

Field	Value
Read-only CometBFT RPC	`https://rpc-mainnet.litho.ai/status` and allowlisted query routes
Explorer	`https://lithoscan.ai`; production cutover accepted 2026-07-31
Public P2P peers	`76cadd27f507c401f58c1335c3f5ece39412f179@31.97.39.146:27056`, `94aa07934bc0614134056bcfe90feb0d214a6e66@72.60.177.106:27056`
Fixed native supply	`1,000,000,000 LITHO`
Measured block interval	0.525 seconds average over heights 177439–178439 on 2026-07-29

Integration position

For exchange custody, the recommended primary integration is the EVM interface:

- use `0x` deposit addresses;
- sign EIP-155 transactions with chain ID `9005`;
- maintain amounts as integer base units with 18 decimals;
- detect native deposits from canonical blocks and successful receipts;
- detect LEP100 deposits from the exact allowlisted contract's `Transfer` logs;
- operate an internal full node and deposit indexer; and
- keep node RPC private even if P2P is public.

Cosmos `litho1...` accounts map to the same 20-byte account space, but an exchange should not mix EVM and Bech32 deposit formats unless it has explicitly tested conversion, signing, accounting, and withdrawal behavior for both.

Numbered exchange questionnaire response

2. Digital coin name and abbreviation

Name: **Lithosphere**. Abbreviation and native ticker: **LITHO**.

3. Block explorer address

The production mainnet explorer is `https://lithoscan.ai`. Its public cutover, chain-identity checks, smoke tests, synchronization monitoring, and rollback closeout passed on 2026-07-31. Do not submit a testnet explorer as the mainnet explorer.

4. GitHub address

Current infrastructure/source repository: `https://github.com/BrewCodeDev/lithosphere-dev-infra`.

Repository access may be restricted. A public, immutable release/tag containing the mainnet binary source, fixed-supply patch, genesis, build instructions, and checksums is an external-submission gate.

5. Node installation documents

See MAINNET_EXCHANGE_NODE_GUIDE.pdf. It covers binary verification, genesis installation, peer configuration, ports, systemd, synchronization, and health checks.

6. Required disk size

Recommended production specification for an exchange wallet node:

Node profile	CPU	RAM	Disk
Pruned full node plus external deposit indexer	8 vCPU	32 GiB	500 GB NVMe SSD
Archive/historical-state node	8–16 vCPU	32–64 GiB	Start at 1 TB NVMe and monitor growth

These are capacity recommendations, not a fixed protocol minimum. The chain is new and disk growth is workload-dependent. Keep at least 30% free space and alert at 70%, 80%, and 90% utilization.

7. Mainnet block data or snapshots

No public, checksum-pinned mainnet data snapshot URL is approved at this time. The canonical genesis is available at: `https://rpc-mainnet.litho.ai/genesis.json`.

Until an official snapshot is published, initialize from genesis and block-sync from the two public peers. Never use a Makalu, Kamet, AWS, or third-party snapshot for `lithosphere_9005-1`.

8. Block production rate

A live 1,000-block sample from height `177439` to `178439`, collected on 2026-07-29, averaged **0.525 seconds per block** with a median of **0.525 seconds**. Treat `~0.5 seconds` as the observed operating interval, not a permanent protocol guarantee.

9. Transaction precision

Native LITHO has 18 decimals. The smallest unit is:

`0.000000000000000001 LITHO = 1 ulitho`.

Store and calculate amounts as integers in `ulitho`; never use binary floating point. LEP100 decimals are contract-specific and must be queried using `decimals()` for every listed contract. The reference LEP100 implementation recommends 18 decimals but does not force it.

10. Public API URL

- EVM JSON-RPC: `https://rpc-mainnet.litho.ai`
- EVM WebSocket: `wss://rpc-mainnet.litho.ai/websocket`
- Cosmos REST: `https://api-mainnet.litho.ai`
- gRPC TLS: `grpc-mainnet.litho.ai:9090`
- Read-only CometBFT status: `https://rpc-mainnet.litho.ai/status`

Public endpoints are rate-limited and should not be the exchange's sole wallet backend. Operate a private full node for production deposits and withdrawals.

11. Memo support

Cosmos transactions support a transaction memo of up to **256 characters**. Ethereum-style native transfers and LEP100 transfers do not have a standardized exchange memo/tag field. EVM transaction `data` must not be treated as a portable deposit memo.

Recommended exchange design: allocate a unique `0x` deposit address per user. Only offer memo-based Cosmos deposits if that separate flow has been fully implemented and tested.

12. Creating accounts

The chain is account-based and does not require an on-chain account-creation transaction. Generate a secp256k1 key securely offline; the 20-byte public-key derived address becomes usable immediately and appears on chain after its first transaction or balance. Use an HSM/MPC custody platform for exchange keys.

Examples and address rules are in the API and node guides. Never send seed phrases or private keys to a node, explorer, support channel, or API.

13. Configuration-file directory

For the recommended exchange deployment:

```
/var/lib/litho-mainnet-9005-exchange/
├── config/
│   ├── app.toml
│   ├── client.toml
│   ├── config.toml
│   ├── genesis.json
│   └── node_key.json
└── data/
```

The home is selected with `--home`. An exchange full node does not need a validator consensus key.

14. Customizing RPC ports and block-data directory

Set the node home with `--home /path`. Configure:

- CometBFT RPC: `[rpc].laddr` in `config/config.toml`;
- P2P: `[p2p].laddr` in `config/config.toml`;
- CometBFT database: top-level `db_dir` in `config/config.toml`;
- REST: `[api].address` in `config/app.toml`;
- gRPC: `[grpc].address` in `config/app.toml`;
- EVM HTTP/WS: `[json-rpc].address` and `ws-address` in `config/app.toml`.

Bind wallet APIs to loopback or a private network. Expose only P2P externally.

15. RPC and SDK documentation

Project-specific examples: MAINNET_EXCHANGE_API_REFERENCE.pdf.

Upstream interfaces:

- Ethereum JSON-RPC: `https://ethereum.org/developers/docs/apis/json-rpc/`
- ethers v6: `https://docs.ethers.org/v6/`
- Cosmos SDK v0.50 transactions: `https://docs.cosmos.network/sdk/v0.50/learn/advanced/transactions`
- CometBFT v0.38 RPC: `https://docs.cometbft.com/v0.38/spec/rpc/`
- OpenZeppelin ERC20: `https://docs.openzeppelin.com/contracts/5.x/api/token/erc20`

16. API instructions

See MAINNET_EXCHANGE_API_REFERENCE.pdf for copyable requests, response conditions, and LEP100 log decoding.

16.1 Account validation

For EVM deposits, validate a strict 20-byte hex address (`0x` plus 40 hex characters), reject the zero address, and use an EIP-55-aware address library. Optionally call `eth_getCode`; `0x` indicates an externally owned address at that block, while non-empty code indicates a contract.

For Cosmos addresses, decode Bech32, require HRP `litho`, and require a 20-byte payload. Do not accept `lithovaloper...` validator-operator addresses as normal deposit addresses.

16.2 Best-height API

- EVM: `eth_blockNumber`
- CometBFT: `GET https://rpc-mainnet.litho.ai/status` and read `result.sync_info.latest_block_height`
- Private REST node: `GET /cosmos/base/tendermint/v1beta1/blocks/latest`

16.3 Account-history API

Standard EVM JSON-RPC has no reliable "all transactions by address" method. The exchange must index blocks, receipts, and token logs into its own database. `eth_getLogs` is suitable for allowlisted LEP100 contracts in bounded ranges.

Cosmos transactions can be queried by hash through `/cosmos/tx/v1beta1/txs/{hash}` and by indexed events on a private node. Lithoscan is available for public browsing and secondary verification, but an exchange must operate its own node and indexer for production accounting.

16.4 Detailed transaction information

For EVM transactions, query both `eth_getTransactionByHash` and `eth_getTransactionReceipt`. Also load the referenced block with `eth_getBlockByNumber` or `eth_getBlockByHash` and verify its hash.

For Cosmos transactions, query `/cosmos/tx/v1beta1/txs/{hash}` and require `tx_response.code == 0`.

16.5 Preventing fraudulent or incorrect deposit credit

Never credit from a pending transaction, calldata alone, or an explorer page.

For native LITHO:

1. Require chain ID `9005` from the node.
2. Fetch the transaction and receipt independently.
3. Require receipt `status == 0x1`.
4. Require the transaction `to` to equal the assigned deposit address.
5. Decode `value` as an integer number of `ulitho`.
6. Fetch the canonical block and require its hash to equal the receipt's `blockHash` and that it contains the transaction hash.
7. Wait for the approved operational confirmation count and ensure the chain continues advancing.
8. Credit idempotently by `(chain_id, transaction_hash)`.

For LEP100:

1. Require the exact allowlisted mainnet token contract address.
2. Require a successful receipt and canonical block as above.
3. Find a `Transfer(address,address,uint256)` log emitted by that contract.
4. Require the indexed `to` topic to equal the assigned deposit address.

5. Decode the log's `value` using that contract's confirmed decimals.
6. Credit idempotently by `(chain_id, transaction_hash, log_index)` because one transaction can emit multiple transfers.

An exchange-operated node should be compared with at least one independent public endpoint. Pause deposits automatically on chain-ID mismatch, stale height, conflicting block hash, or validator/consensus alerts.

16.6 Offline signing and online broadcast

Recommended EVM flow:

1. Online system obtains `nonce`, fee data, destination, amount, and gas limit.
2. Offline/HSM system validates chain ID `9005`, destination, amount, nonce, and policy; it then signs the complete EIP-155 transaction.
3. Only the signed raw bytes return online.
4. Online system broadcasts with `eth_sendRawTransaction`.
5. Track the returned hash through receipt inclusion and the deposit/withdrawal finality policy.

The private key never reaches the RPC node. See the API reference for an ethers v6 example and a Cosmos `SIGN_MODE_DIRECT` alternative.

16.7 Account balance API

- Native EVM balance: `eth_getBalance(address, "latest")`
- Native Cosmos balance: `/cosmos/bank/v1beta1/balances/{litho_address}/by_denom?denom=ulitho`
- LEP100 balance: `eth_call` to `balanceOf(address)` on the exact token contract

All returned values are base-unit integers.

17. Measures to prevent chain forks

- pin chain ID, genesis checksum, and reviewed binary checksum;
- connect to both official sentries and compare against an independent RPC;
- run NTP/time synchronization and monitor peers, height, and block hashes;
- never reuse a validator consensus key or validator signing state on an exchange node;
- keep the node binary and configuration under change control;
- halt deposits on conflicting canonical hashes or stalled consensus; and
- follow coordinated network-upgrade notices before changing binaries.

CometBFT provides deterministic BFT finality, not longest-chain PoW finality. However, the current mainnet validator set contains one validator, so the exchange must treat operator compromise or catastrophic recovery as an additional governance/operational risk.

18. Account restore and recovery recommendations

- Use HSM/MPC or offline encrypted seed custody with dual control.
- Back up derivation metadata, account inventory, signing policies, and address mappings separately from node data.
- Test recovery with a watch-only environment before enabling withdrawals.
- Restore accounts from keys; restore node state from an approved snapshot or resync. Node data is not the source of account ownership.
- Never restore a validator key or `priv_validator_state.json` onto an exchange node.

- Reconcile all deposit addresses and balances after recovery before reopening withdrawals.

19. Coin symbol

`LITHO`.

20. Officially recognized wallet

Thanos Wallet is the Lithosphere ecosystem wallet at `https://thanos.fi`, and MetaMask-compatible wallets can add EVM chain `9005` manually. At the time of this draft, Thanos' public material identifies Makalu rather than the new `9005` mainnet, so mainnet support must be explicitly confirmed before the wallet is submitted to an exchange as production-ready.

Manual EVM configuration:

```
Network name: Lithosphere Mainnet
RPC URL: https://rpc-mainnet.litho.ai
Chain ID: 9005
Currency symbol: LITHO
Explorer: https://lithoscan.ai
```

21. Same currency on other networks and bridge audit

No cross-chain representation of mainnet `9005` LITHO or LEP100 token is approved for exchange listing in this package. MultX, Bridge, and Swap remain disabled on mainnet. The repository contains an audit RFQ and internal test evidence, not a completed independent bridge audit report. Therefore there is no bridge-audit link that can responsibly be supplied yet.

Do not treat any Makalu/Kamet token, old wLITHO contract, or destination-chain wrapped asset as interchangeable with mainnet LITHO.

22. PoW and 51% attack

Not applicable. Lithosphere mainnet uses proof-of-stake CometBFT consensus, not proof of work. There are no miners or top-five miner addresses. The current validator set can be queried from `https://rpc-mainnet.litho.ai/validators`; at the time of review it contained one validator.

23. Common transfer types

- EVM native LITHO transfer (`value` transfer)
- Cosmos bank `MsgSend`
- Cosmos bank `MsgMultiSend`
- LEP100 `transfer(address,uint256)`
- LEP100 `transferFrom(address,address,uint256)` after allowance

The reference LEP100 contract has no standard `batch`, `batchAll`, reflection, fee-on-transfer, rebase, or proxy-upgrade behavior. Exchanges must still review each exact contract they list.

24. Rollback and irreversible blocks

Committed CometBFT blocks have deterministic finality and do not normally experience probabilistic reorganizations. There is no protocol concept of "safe after N PoW blocks." Operationally, exchanges should still wait a confirmation buffer and watch continued height advancement.

Provisional integration recommendation: **20 blocks** for normal deposits and **100 blocks** for high-value deposits until the client and exchange approve a formal policy. At the measured rate, those are roughly 10.5 and 52.5 seconds. These values are recommendations, not yet an officially signed listing policy.

25. Transaction-pool timeout

The deployed configuration does not set a time-based mempool TTL. A valid transaction may remain pending until included, invalidated during recheck, replaced according to nonce/fee rules, evicted by capacity pressure, or removed when the node restarts. Exchange systems must implement their own pending timeout, nonce reconciliation, safe rebroadcast, and replacement policy.

26. Rent or minimum account reservation

There is no account rent, storage rent, or mandatory minimum balance for an ordinary EVM/Cosmos account. Users only need sufficient LITHO to pay transaction fees. Staking and governance actions have their own amounts but do not impose a general account reserve.

27. UTXO failure determination

Not applicable. Lithosphere is account-based. Validate nonce/sequence, balance, gas, fee, destination, chain ID, signature, and—when calling a contract—the simulation result. For LEP100 withdrawals, also validate token balance and allowance when `transferFrom` is used.

28. Mainnet transfer start time

The sealed height-1 timestamp is `2026-07-27T17:00:00Z`; continuous live block production began at height 2 on `2026-07-28T04:26:52.822404135Z`. Native transfers are technically enabled by the chain.

The official exchange deposit/withdrawal opening time is **TBD** and should be announced only after the explorer, monitoring, confirmation policy, wallet testing, and exchange release checklist are approved.

29. Must a wallet node expose a port? Is NAT acceptable?

The wallet's RPC, REST, gRPC, EVM, and metrics ports should remain private. Only the P2P port needs network connectivity. Outbound-only P2P can synchronize, so NAT is acceptable. For stable inbound peer connectivity, forward one TCP P2P port and configure the advertised external address. Never port-forward the custody RPC directly to the Internet.

30. Address rules

Address type	Rule
EVM account/contract	`0x` plus 40 hexadecimal characters; total length 42; 20-byte payload; EIP-55 checksum recommended
Cosmos account	Bech32 HRP `litho`; typical length 44; decoded payload must be 20 bytes
Validator operator	Bech32 HRP `lithovaloper`; not a normal exchange deposit address

Reject malformed mixed-case EVM checksums, the zero address, wrong Bech32 HRP, wrong payload length, and all addresses from other networks.

31. Node-IP whitelist for synchronization

No node-IP whitelist is required for normal mainnet P2P synchronization. Use the two published persistent peers. If an exchange requests a dedicated peer or higher service level, coordinate it separately; do not expose custody RPC or ask for a validator private peer.

32. Community notice regarding MEXC deposit addresses

Lithosphere is not mined, so "mining deposit" is not applicable. The community notice should state:

> Do not use a MEXC or any other exchange deposit address for validator rewards, staking ownership, token distribution contracts, faucets, mining-like payouts, or long-term custody. Send only the exact asset on the exchange-supported Lithosphere network after deposits are officially open. Unsupported networks, tokens, or operational payouts may not be credited or recoverable.

External references

- Ethereum JSON-RPC: `https://ethereum.org/developers/docs/apis/json-rpc/`
- ethers v6: `https://docs.ethers.org/v6/`
- Cosmos SDK v0.50 transactions: `https://docs.cosmos.network/sdk/v0.50/learn/advanced/transactions`
- CometBFT v0.38 RPC: `https://docs.cometbft.com/v0.38/spec/rpc/`
- CometBFT production guidance: `https://docs.cometbft.com/v0.38/core/running-in-production`
- OpenZeppelin ERC20: `https://docs.openzeppelin.com/contracts/5.x/api/token/erc20`

LITHO Mainnet Exchange Node Guide

Last reviewed: 2026-07-29

Audience: exchange wallet, custody, and infrastructure teams operating a non-validating `lithosphere_9005-1` full node.

Status: technically complete except for the public signed binary-release URL and public snapshot URL. Do not substitute an older Lithosphere binary, genesis, or testnet snapshot.

1. Immutable network inputs

Input	Approved value
Cosmos chain ID	`lithosphere_9005-1`
EVM chain ID	`9005` (`0x232d`)
Binary	Linux x86_64 `lithod`, Evmos v20-derived fixed-supply build
Binary SHA-256	`0546677a9cf3a7f458797b65181a46f21c89185933e832d89ce728a144fd258c`
Genesis URL	`https://rpc-mainnet.litho.ai/genesis.json`
Genesis SHA-256	`13e4875b4a9dddc63bdfbd4968c7265f9bbc49218b59c5b49231a56fa313046f`
Base denomination	`ulitho` (`10^18 ulitho = 1 LITHO`)
CometBFT version reported live	`0.38.12`

The binary contains a consensus-critical fixed-supply patch. An exchange must verify the exact release artifact and checksum before synchronization. The public binary URL is currently `TBD`; obtain the artifact through the approved release channel, not chat or an unverified file share.

2. Recommended production host

Resource	Pruned exchange node	Archive/historical node
CPU	8 vCPU	8–16 vCPU
RAM	32 GiB	32–64 GiB
Disk	500 GB NVMe SSD	1 TB NVMe SSD initially
Network	Stable 100 Mbps+	Stable 100 Mbps+
OS	Ubuntu 24.04/26.04 LTS x86_64	Ubuntu 24.04/26.04 LTS x86_64

Use a dedicated filesystem or volume for the node home. Monitor disk growth and keep at least 30% free. A separate database should hold the exchange's deposit index; node state is not an accounting database.

3. Port policy

Recommended local ports are standard and may be changed:

Service	Default	Exposure
CometBFT P2P	TCP `26656`	Public or outbound-only through NAT
CometBFT RPC	TCP `26657`	Loopback/private only
REST/LCD	TCP `1317`	Loopback/private only
gRPC	TCP `9090`	Loopback/private only
EVM JSON-RPC	TCP `8545`	Loopback/private only
EVM WebSocket	TCP `8546`	Loopback/private only
Prometheus	TCP `26660`	Monitoring network only

Do not expose custody RPCs directly to the Internet. Place authentication, rate limiting, request-size limits, and network ACLs in front of any shared internal RPC.

4. Install the approved binary

Create a dedicated user and home:

```
sudo useradd --system --home /var/lib/litho-mainnet-9005-exchange \
  --shell /usr/sbin/nologin litho
sudo install -d -o litho -g litho -m 0750 \
  /var/lib/litho-mainnet-9005-exchange
```

Install the artifact received from the approved release channel:

```
sudo install -o root -g root -m 0755 ./lithod-mainnet-9005 \
  /usr/local/bin/lithod-mainnet-9005

echo '0546677a9cf3a7f458797b65181a46f21c89185933e832d89ce728a144fd258c  /usr/local/bin/lithod-mainnet-9005' \
  | sha256sum --check

/usr/local/bin/lithod-mainnet-9005 version --long
```

Stop if the checksum differs.

Source-build fallback

The current source repository is `https://github.com/BrewCodeDev/lithosphere-dev-infra`. Its `bin/build-lithod.sh` checks out Evmos `v20.0.0`, applies the fixed-supply patch, rebrands the chain, runs focused tests, and builds Linux x86_64 `lithod`.

Use a signed release tag/commit supplied by the project. Locally built binaries may not be byte-for-byte identical because build metadata/toolchains can vary; record the build environment, source commit, test output, and resulting hash.

5. Initialize the node

```
export LITHO_HOME=/var/lib/litho-mainnet-9005-exchange

sudo -u litho /usr/local/bin/lithod-mainnet-9005 init exchange-node-01 \
  --chain-id lithosphere_9005-1 \
  --home "$LITHO_HOME"

sudo -u litho curl --fail --silent --show-error --location \
  https://rpc-mainnet.litho.ai/genesis.json \
  --output "$LITHO_HOME/config/genesis.json"

echo '13e4875b4a9dddc63bdfbd4968c7265f9bbc49218b59c5b49231a56fa313046f  /var/lib/litho-mainnet-9005-exchange/config/genesis.json' \
  | sha256sum --check

/usr/local/bin/lithod-mainnet-9005 validate-genesis \
  "$LITHO_HOME/config/genesis.json"
```

`init` creates local node identity files. Those are not the network validator's consensus key. Never copy a validator `priv_validator_key.json` or signing state onto an exchange node.

6. Configure P2P and RPC

Edit `$LITHO_HOME/config/config.toml`.

Required identity and peer settings:

```
moniker = "exchange-node-01"
db_backend = "goleveldb"
db_dir = "data"

[rpc]
laddr = "tcp://127.0.0.1:26657"
unsafe = false

[p2p]
laddr = "tcp://0.0.0.0:26656"
external_address = ""
seeds = ""
persistent_peers =
"76cadd27f507c401f58c1335c3f5ece39412f179@31.97.39.146:27056,94aa07934bc0614134056bcfe90feb0d214a6e66@72.60.177.106:27056"
pex = true

[tx_index]
indexer = "kv"

[instrumentation]
prometheus = true
prometheus_listen_addr = "127.0.0.1:26660"
```

If inbound P2P is enabled behind NAT, forward TCP `26656` and set `external_address = "PUBLIC_IP:26656"`. Outbound-only P2P works without it.

Do not change consensus timeout settings on a non-validator unless directed by a coordinated network release. Do not enable state sync without a project- published trust height, trust hash, and compatible snapshot service.

7. Configure application APIs and storage

Edit `$LITHO_HOME/config/app.toml`:

```
minimum-gas-prices = "0.0001ulitho"
pruning = "default"
pruning-keep-recent = "0"
pruning-interval = "0"

[api]
enable = true
swagger = false
address = "tcp://127.0.0.1:1317"
enabled-unsafe-cors = false

[grpc]
enable = true
address = "127.0.0.1:9090"

[json-rpc]
enable = true
address = "127.0.0.1:8545"
ws-address = "127.0.0.1:8546"
api = "eth,net,web3"
allow-unprotected-txs = false
enable-indexer = false

[memiavl]
enable = true
```

For archive application state, use `pruning = "nothing"` and provision more disk. CometBFT block and transaction-index databases still grow separately. The exchange must maintain its own address/deposit index even on an archive node.

MemIAVL must remain enabled to match the live chain. Do not copy storage settings from an older Makalu/Kamet guide.

8. Install systemd service

Create `/etc/systemd/system/lithod-mainnet-9005-exchange.service`:

```
[Unit]
Description=LITHO Mainnet 9005 Exchange Full Node
After=network-online.target
Wants=network-online.target

[Service]
User=litho
Group=litho
Type=simple
ExecStart=/usr/local/bin/lithod-mainnet-9005 start --home /var/lib/litho-mainnet-9005-exchange
Restart=on-failure
RestartSec=5
LimitNOFILE=1048576
TimeoutStopSec=90
KillSignal=SIGINT

[Install]
WantedBy=multi-user.target
```

Activate it:

```
sudo systemctl daemon-reload
sudo systemctl enable --now lithod-mainnet-9005-exchange
sudo journalctl -u lithod-mainnet-9005-exchange -f
```

9. Synchronization

No approved public snapshot is available. Genesis/block sync is the supported bootstrap path at this time.

```
curl -fsS http://127.0.0.1:26657/status | jq '{
  network: .result.node_info.network,
  height: .result.sync_info.latest_block_height,
  block_time: .result.sync_info.latest_block_time,
  catching_up: .result.sync_info.catching_up
}'

curl -fsS http://127.0.0.1:26657/net_info | jq '.result.n_peers'
```

Synchronization is complete only when:

- network is `lithosphere_9005-1`;
- height is close to two independent remote endpoints;
- `catching_up` is `false`;
- latest block time is current; and
- the height advances across repeated samples.

10. Exchange acceptance checks

```
# EVM chain ID: expected 0x232d
curl -fsS http://127.0.0.1:8545 \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_chainId","params":[]}'

# Latest height
curl -fsS http://127.0.0.1:8545 \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_blockNumber","params":[]}'

# Local genesis and binary integrity
sha256sum /var/lib/litho-mainnet-9005-exchange/config/genesis.json
sha256sum /usr/local/bin/lithod-mainnet-9005

# Public comparison
curl -fsS https://rpc-mainnet.litho.ai/status | jq '.result.sync_info'
```

Before opening deposits, also test account derivation, native deposits, LEP100 log detection, withdrawal signing, nonce recovery, service restart, database restore, alert routing, and idempotent replay from an earlier height.

11. Backup and recovery

Back up:

- `config/node_key.json` if stable node identity matters;
- configuration and service files;
- the exchange deposit-index database;
- wallet derivation metadata and custody policies through the HSM/MPC system;
- exact binary, source release reference, genesis, and checksums.

Do not treat the node `data/` directory as a wallet-key backup. A non-validator node may be rebuilt from an approved snapshot or genesis. Never use `unsafe-reset-all` without a reviewed rebuild plan and a verified accounting checkpoint.

12. Operational alerts

Page on:

- no new block for more than two minutes;
- `catching_up=true` after initial sync;
- chain-ID or genesis mismatch;
- local/public canonical block-hash mismatch;
- zero peers or both official peers unavailable;
- disk over 80%, file descriptors, memory pressure, or restart loop;
- RPC latency/error-rate threshold breach;
- pending withdrawal nonce gap; and
- certificate expiry for any proxy operated by the exchange.

LITHO Mainnet Exchange API Reference

Last reviewed: 2026-07-29

This document provides exchange-oriented examples for native LITHO and LEP100 assets on EVM chain `9005`. Replace placeholders and use a private exchange node for production. Public endpoints are intended for integration and independent comparison, not as the sole custody backend.

1. Endpoints

Interface	Public endpoint
EVM JSON-RPC	`POST https://rpc-mainnet.litho.ai`
EVM WebSocket	`wss://rpc-mainnet.litho.ai/websocket`
Cosmos REST	`https://api-mainnet.litho.ai`
gRPC TLS	`grpc-mainnet.litho.ai:9090`
Read-only CometBFT	`GET https://rpc-mainnet.litho.ai/{allowlisted_route}`

The public EVM proxy is limited to 25 requests/second/IP with a bounded burst; REST is limited to 30 requests/second/IP. A single `eth_getLogs` query is also bounded by node-side block-range and result caps. Paginate all scans.

2. JSON-RPC request helper

```
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"METHOD","params":[]}'
```

Every client must reject a response containing `error`, an unexpected `id`, or an invalid `jsonrpc` version. Quantities are hexadecimal integers, not decimal strings.

3. Network and node health

EVM chain ID

```
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_chainId","params":[]}'
```

Expected `result`: `0x232d`.

Best EVM height

```
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_blockNumber","params":[]}'
```

Convert the hex result to an integer.

Cosmos identity and height

```
curl -fsS https://rpc-mainnet.litho.ai/status | jq '{
  chain_id: .result.node_info.network,
  height: .result.sync_info.latest_block_height,
  block_hash: .result.sync_info.latest_block_hash,
  block_time: .result.sync_info.latest_block_time,
  catching_up: .result.sync_info.catching_up
}'
```

Require chain ID `lithosphere_9005-1`, current time, advancing height, and `catching_up=false`.

Peer and validator state

```
curl -fsS https://rpc-mainnet.litho.ai/net_info | jq '.result.n_peers'
curl -fsS https://rpc-mainnet.litho.ai/validators | jq '.result'
```

The validator response contained one validator at review time. Alert on any unexpected validator-set or voting-power change.

4. Address and account validation

EVM syntax

Use a maintained Ethereum address library. A valid address is 20 bytes, normally rendered as `0x` plus 40 hex characters. Require a valid EIP-55 checksum when mixed case is supplied. Reject `0x0000000000000000000000000000000000000000`.

Determine whether an EVM address has contract code

```
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_getCode","params":["0xADDRESS","latest"]}'
```

`0x` normally means an externally owned account at the selected block. A non-empty result means contract code is present. Contract accounts are valid on chain, but an exchange may reject them as deposit destinations according to its withdrawal policy.

Cosmos account

```
curl -fsS \
  https://api-mainnet.litho.ai/cosmos/auth/v1beta1/accounts/litho1ADDRESS \
  | jq '.account'
```

An HTTP not-found response does not necessarily make an address invalid: a syntactically valid account may not yet have on-chain state. Validate Bech32 locally first.

5. Balances and nonces

Native EVM balance

```
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_getBalance","params":["0xADDRESS","latest"]}'
```

Result is an integer number of `ulitho` encoded as hex.

EVM transaction count / nonce

```
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_getTransactionCount","params":["0xADDRESS","pending"]}'
```

Use `pending` for constructing a new withdrawal and `latest` for confirmed state. The exchange must serialize withdrawals per sender or use a durable nonce allocator.

Cosmos native balance

```
curl -fsS \
  'https://api-mainnet.litho.ai/cosmos/bank/v1beta1/balances/litho1ADDRESS/by_denom?denom=ulitho' \
  | jq '.balance'
```

Amount is a decimal-string integer in `ulitho`.

LEP100 balance

The `balanceOf(address)` selector is `0x70a08231`. Append the address left-padded to 32 bytes:

```
TOKEN=0xTOKEN_CONTRACT
ACCOUNT_WITHOUT_0X=0000000000000000000000000000000000000000

curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data "{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"eth_call\",\"params\":
[{\"to\":\"$TOKEN\",\"data\":\"0x70a08231000000000000000000000000$ACCOUNT_WITHOUT_0X\"},\"latest\"]}"
```

Use an ABI library rather than hand-building calldata in production.

6. Blocks and transaction details

Load block with full transactions

```
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_getBlockByNumber","params":["0xBLOCK_NUMBER",true]}'
```

Store block number, block hash, parent hash, timestamp, and transaction hashes. Before credit, reload the block by number and confirm its hash has not changed.

Transaction object

```
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_getTransactionByHash","params":["0xTX_HASH"]}'
```

Transaction receipt

```
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_getTransactionReceipt","params":["0xTX_HASH"]}'
```

A pending/unknown transaction returns `null`. A mined transaction must have a receipt. Require `status == 0x1`; `0x0` means EVM execution failed. Also verify `transactionHash`, `blockHash`, `blockNumber`, `to`, and all relevant logs.

Cosmos transaction by hash

```
curl -fsS \
  https://api-mainnet.litho.ai/cosmos/tx/v1beta1/txs/TX_HASH \
  | jq '.tx_response'
```

Require `code == 0`. Do not credit a Cosmos transfer based only on raw message contents when execution failed.

7. Account history and deposit indexing

Ethereum JSON-RPC does not define a complete transactions-by-address endpoint. The production exchange indexer should:

1. persist its last fully processed height;
2. read each canonical block by height;
3. store native transactions and receipts relevant to exchange addresses;
4. query allowlisted token logs in bounded ranges;
5. wait the approved confirmation buffer;
6. credit with an idempotency key; and
7. replay safely from an earlier checkpoint after recovery.

Do not use explorer search results as an accounting source.

8. LEP100 `Transfer` log scanning

Event signature:

```
Transfer(address,address,uint256)
topic0 = 0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef
topic1 = from address, left-padded to 32 bytes
topic2 = to address, left-padded to 32 bytes
data   = amount as uint256 base units
```

Filter one exact token and recipient:

```
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{
    "jsonrpc":"2.0",
    "id":1,
    "method":"eth_getLogs",
    "params":[{
      "fromBlock":"0xSTART",
      "toBlock":"0xEND",
      "address":"0xALLOWLISTED_TOKEN",
      "topics":[
        "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef",
        null,
        "0x000000000000000000000000DEPOSIT_ADDRESS_WITHOUT_0X"
      ]
    }]
  }'
```

Credit by `(chain_id, transaction_hash, log_index)`. Never infer a token from symbol alone; symbols and names are not unique.

9. LEP100 metadata calls

Function	Selector	Expected ABI result
`name()`	`0x06fdde03`	string
`symbol()`	`0x95d89b41`	string
`decimals()`	`0x313ce567`	uint8
`totalSupply()`	`0x18160ddd`	uint256

Example:

```
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_call","params":[{"to":"0xTOKEN_CONTRACT","data":"0x313ce567"},"latest"]}'
```

Decode ABI output using ethers/viem/web3. Verify metadata once at listing and monitor contract code hash afterward.

10. Gas and fee queries

```
# Legacy-compatible gas price
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_gasPrice","params":[]}'

# Estimate gas for an exact transaction object
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_estimateGas","params":[{"from":"0xFROM","to":"0xTO","value":"0xVALUE"}]}'
```

Do not hard-code the observed gas price. Estimate against current state, apply an exchange policy margin, and enforce a maximum fee.

11. Offline EVM signing and online broadcast

The online system may prepare an unsigned EIP-1559 transaction:

```
{
  "type": 2,
  "chainId": 9005,
  "nonce": 12,
  "to": "0xRECIPIENT",
  "value": "1000000000000000000",
  "gasLimit": "21000",
  "maxFeePerGas": "2000000000",
  "maxPriorityFeePerGas": "0",
  "data": "0x"
}
```

The offline/HSM zone must independently verify every field. A simplified ethers v6 illustration is:

```
import { Wallet } from "ethers";
import unsigned from "./approved-unsigned-transaction.json" with { type: "json" };

// Illustration only. Production exchanges should use an HSM/MPC signer and
// must not load raw private keys from environment variables.
const signer = new Wallet(process.env.OFFLINE_DEMO_PRIVATE_KEY);
if (Number(unsigned.chainId) !== 9005) throw new Error("wrong chain ID");
const signedRawTransaction = await signer.signTransaction(unsigned);
console.log(signedRawTransaction);
```

Broadcast signed bytes from the online node:

```
curl -fsS http://127.0.0.1:8545 \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_sendRawTransaction","params":["0xSIGNED_RAW_TRANSACTION"]}'
```

Recompute the transaction hash locally, compare it with the RPC response, and then monitor the receipt. Never reassign a nonce without reconciling pending and confirmed transactions.

12. Offline Cosmos signing alternative

EVM is the recommended exchange integration. If Cosmos `litho1` withdrawals are supported, pin chain ID, account number, sequence, fee, memo, and denom:

```
# Online/watch-only construction
lithod tx bank send litho1FROM litho1TO 1000000000000000000ulitho \
  --chain-id lithosphere_9005-1 \
  --account-number ACCOUNT_NUMBER \
  --sequence SEQUENCE \
  --fees 1000ulitho \
  --memo 'EXCHANGE_WITHDRAWAL_ID' \
  --generate-only > unsigned.json

# Offline signer
lithod tx sign unsigned.json \
  --offline \
  --from CUSTODY_KEY \
  --chain-id lithosphere_9005-1 \
  --account-number ACCOUNT_NUMBER \
  --sequence SEQUENCE \
  --output-document signed.json

# Online broadcast through the exchange node
lithod tx broadcast signed.json \
  --node tcp://127.0.0.1:26657 \
  --broadcast-mode sync
```

Query the transaction until included and require `code == 0`.

13. Deposit-credit algorithm

For each candidate deposit:

1. Ensure local and independent nodes agree on both chain IDs and are advancing.
2. Require a canonical block and successful execution result.
3. Match the exact assigned recipient.
4. Match native LITHO or the exact allowlisted LEP100 contract.
5. Decode integer base units and enforce configured decimals.
6. Apply sanctions/risk and minimum-deposit policies.
7. Wait the approved confirmation buffer.
8. Recheck the block hash before credit.
9. Credit exactly once using a durable idempotency key.
10. Retain raw block, transaction, receipt/log, and parsing evidence.

Pause crediting on stale blocks, conflicting hashes, parser errors, token-code changes, or an unapproved validator-set/binary upgrade.

14. Useful upstream references

- Ethereum JSON-RPC: `https://ethereum.org/developers/docs/apis/json-rpc/`
- ethers v6 provider and signing APIs: `https://docs.ethers.org/v6/`
- Cosmos SDK transactions: `https://docs.cosmos.network/sdk/v0.50/learn/advanced/transactions`
- CometBFT RPC: `https://docs.cometbft.com/v0.38/spec/rpc/`

LEP100 Mainnet Exchange Integration

Last reviewed: 2026-07-29

Status: **standard documented; mainnet token registry pending**. No LEP100 contract address is approved for exchange listing on EVM chain `9005` in this repository at the time of review. Testnet addresses must never be reused as mainnet asset identifiers.

1. What LEP100 means on mainnet

The current reference implementation is an EVM fungible token built from OpenZeppelin Contracts 5.x:

- `ERC20`
- `ERC20Burnable`
- `Ownable`

Reference source: `contracts/contracts/LEP100Token.sol`.

Property	Reference behavior
Solidity version	`0.8.24`
Source SHA-256	`d3153fd7473498c872913eb0cf213dae61cd608048d815cf7211b3cb835dfdc1`
Supply	Fixed at construction; minted to deployer
Decimals	Immutable constructor value; query per token
Transfer	Standard ERC20 `transfer` / `transferFrom`
Burn	Holder `burn`; allowance-based `burnFrom`
Mint after construction	Not present
Pause/blacklist/tax/rebase	Not present
Proxy/upgrades	Not present in reference contract
Permit	Not present; no `ERC20Permit` inheritance
Gas coin	Native LITHO

`Ownable` does not grant a hidden mint or freeze function because the reference contract defines no owner-only operational methods. Ownership can still be transferred or renounced through inherited `Ownable` functions.

Any modified deployment must be reviewed as a separate token contract. A name or symbol containing "LEP100" does not prove conformance.

2. Constructor and amount units

```
constructor(
    string memory name_,
    string memory symbol_,
    uint8 decimals_,
    uint256 totalSupply_
)
```

The constructor treats `totalSupply_` as whole display tokens and internally mints:

`totalSupply_ * 10 ** decimals_` base units.

An exchange must query and store the resulting on-chain `totalSupply()` and `decimals()` values; do not reconstruct them from issuer marketing material.

3. Mainnet listing identifier

The unique asset key must include:

```
network = lithosphere-mainnet
evm_chain_id = 9005
contract_address = 0x...
```

Never identify a token by symbol alone. Symbols and names can collide. Store the checksummed contract address and its bytecode hash.

4. Required listing dossier

Before enabling a LEP100 asset, obtain:

- exact mainnet contract address on chain `9005`;
- deployment transaction hash and deployment block;
- issuer/legal entity and authorized contact;
- name, symbol, decimals, and total supply;
- verified Solidity source, compiler version, optimizer settings, and constructor arguments;
- source and deployed-bytecode hashes;
- ownership address and ownership/renouncement policy;
- distribution and treasury addresses;
- token-specific audit report, if required by the exchange;
- logo/metadata separately from the on-chain identity;
- deposit minimum, withdrawal fee, and confirmation policy; and
- explicit statement whether the contract is the unmodified reference or a modified implementation.

The production explorer is not yet available for source verification. Until it is approved, verification evidence must come from reproducible compilation and direct RPC code comparison.

5. On-chain conformance checks

Run all calls against the exact proposed contract:

Method	Selector	Requirement
`name()`	`0x06fdde03`	ABI string; informational only
`symbol()`	`0x95d89b41`	ABI string; not a unique ID
`decimals()`	`0x313ce567`	uint8; configure accounting exactly
`totalSupply()`	`0x18160ddd`	uint256 base units
`balanceOf(address)`	`0x70a08231`	uint256 base units
`transfer(address,uint256)`	`0xa9059cbb`	standard successful transfer behavior
`allowance(address,address)`	`0xdd62ed3e`	uint256
`approve(address,uint256)`	`0x095ea7b3`	standard approval behavior
`transferFrom(address,address,uint256)`	`0x23b872dd`	allowance-based transfer

Also verify:

1. `eth_getCode` is non-empty at the listing block and current head.
2. The deployment receipt succeeded.
3. Reproducibly compiled runtime bytecode matches the deployed runtime code, accounting for linked/immutable metadata as appropriate.
4. A test transfer produces exactly the expected recipient balance delta and a standard `Transfer` event.
5. No proxy storage slot or delegatecall upgrade mechanism exists unless the exchange has explicitly approved an upgradeable token.
6. No fee-on-transfer, reflection, rebasing, blacklist, pause, or hidden mint behavior exists.

6. Minimal exchange ABI

```
[
  {"type":"function","name":"name","stateMutability":"view","inputs":[],"outputs":[{"type":"string"}]},
  {"type":"function","name":"symbol","stateMutability":"view","inputs":[],"outputs":[{"type":"string"}]},
  {"type":"function","name":"decimals","stateMutability":"view","inputs":[],"outputs":[{"type":"uint8"}]},
  {"type":"function","name":"totalSupply","stateMutability":"view","inputs":[],"outputs":[{"type":"uint256"}]},
  {"type":"function","name":"balanceOf","stateMutability":"view","inputs":[{"name":"account","type":"address"}],"outputs":
[{"type":"uint256"}]},
  {"type":"function","name":"transfer","stateMutability":"nonpayable","inputs":[{"name":"to","type":"address"},
{"name":"value","type":"uint256"}],"outputs":[{"type":"bool"}]},
  {"type":"event","name":"Transfer","anonymous":false,"inputs":[{"name":"from","type":"address","indexed":true},
{"name":"to","type":"address","indexed":true},{"name":"value","type":"uint256","indexed":false}]}
]
```

Use the full verified ABI for security review; this minimal ABI is only for routine exchange balance, transfer, and deposit operations.

7. Deposit detection

The standard event is:

```
Transfer(address indexed from, address indexed to, uint256 value)
topic0 = 0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef
```

For every candidate deposit:

1. Require EVM chain ID `9005`.
2. Require log `address` to equal the exact allowlisted token contract.
3. Require `topic0` to equal the `Transfer` signature.
4. Decode indexed `to` and require the assigned exchange deposit address.
5. Decode `value` as a uint256 integer.
6. Fetch and require receipt `status == 0x1`.
7. Verify receipt block hash against the canonical block.
8. Wait the exchange-approved confirmation buffer.
9. Credit by `(9005, transaction_hash, log_index)`.

Do not credit from input calldata. A reverted transaction can contain valid- looking `transfer` calldata but moves no tokens. A single transaction can emit multiple `Transfer` logs, so transaction hash alone is not a sufficient idempotency key.

8. Withdrawals and token sweeping

Use standard `transfer(recipient, amount)` from the custody address. Before signing:

- validate chain ID `9005`;
- validate recipient address and reject zero address;
- ensure `amount` is an integer base-unit value;
- query current token balance;
- estimate gas against the exact contract call;
- simulate with `eth_call` where custody tooling permits;
- reserve native LITHO for gas; and
- enforce nonce, fee, and withdrawal policy in the offline/HSM signer.

Each token-holding deposit address needs native LITHO to sweep tokens. Plan a controlled gas-funding workflow and prevent gas-dust abuse. Token fees cannot normally be paid in the LEP100 token itself.

9. Decimals and accounting

If `decimals() = d`:

`display amount = base-unit integer / 10^d`.

Use arbitrary-precision integers and decimal formatting. Never use IEEE-754 floating point for balances, credits, or withdrawals. Reject a withdrawal that cannot be represented exactly in the token's configured base units.

Monitor `decimals`, `totalSupply`, bytecode, and ownership state after listing. The reference decimals are immutable, but monitoring detects contract/address substitution and unreviewed implementations.

10. Common transfer types

- Direct holder transfer: `transfer(to, amount)`
- Allowance-based transfer: `approve(spender, amount)` then `transferFrom(from, to, amount)`
- Burn: `burn(amount)`
- Allowance-based burn: `burnFrom(account, amount)`

The reference contract does not contain batch transfer, batch-all, mint, pause, freeze, tax, blacklist, rebase, permit, or proxy upgrade functions.

11. Finality and replay

LEP100 transactions inherit Lithosphere's CometBFT finality. The provisional exchange recommendation is 20 blocks for routine deposits and 100 for high-value deposits, subject to formal client/exchange approval.

The indexer must be able to replay a block range without double crediting. Persist canonical block hashes and revalidate them during recovery. Pause crediting if the node stalls, chain IDs differ, block hashes conflict, or the token code changes.

12. Cross-chain and wrapped-token warning

MultX and the mainnet bridge are disabled. No destination-chain wrapped LEP100 or wrapped LITHO address is approved in this package, and there is no completed independent bridge audit report to provide.

Do not accept:

- Makalu/Kamet LEP100 contract addresses;
- old `wLITHO` testnet contracts;
- an issuer-provided token on Ethereum, BNB, Base, or another network as equivalent to the mainnet asset; or
- bridge mint/burn events as deposits without a separately approved bridge integration and audit.

13. Mainnet registry required before listing

The project must publish a machine-readable mainnet registry containing, for each approved token:

```
{
  "network": "lithosphere-mainnet",
  "chainId": 9005,
  "contract": "0x...",
  "name": "...",
  "symbol": "...",
  "decimals": 18,
  "deploymentTx": "0x...",
  "runtimeCodeHash": "0x...",
  "sourceRelease": "...",
  "status": "active"
}
```

It must be signed or delivered from an authenticated official release channel. Explorer discovery alone must not add an asset to an exchange allowlist.

14. References

- Local reference source: `contracts/contracts/LEP100Token.sol`

- OpenZeppelin Contracts 5.x ERC20: `https://docs.openzeppelin.com/contracts/5.x/api/token/erc20`
- Ethereum JSON-RPC: `https://ethereum.org/developers/docs/apis/json-rpc/`
- Project API examples: MAINNET_EXCHANGE_API_REFERENCE.pdf

LITHO Mainnet Exchange Documentation Release Checklist

Last reviewed: 2026-07-31

Purpose: prevent a technically useful draft from being shared as a final listing package before client-owned production facts are ready.

1. Current disposition

Area	Status	External wording
Base chain	Live and reverified	May be stated as live
EVM/Cosmos node APIs	Live	May be supplied with rate-limit/private-node caveat
Explorer	Live and cutover-accepted	May provide `https://lithoscan.ai` with the exchange's independent-indexing requirement
Public source release	Access/publication pending	Current repository may require access
Signed binary download	URL pending	Supply checksum only with controlled artifact
Public block-data snapshot	Not available	State clearly; sync from genesis
Mainnet LEP100 registry	Not published	Do not list testnet contracts
Official wallet on chain 9005	Compatibility confirmation pending	MetaMask manual config only; qualify Thanos
Mainnet bridge	Disabled	No wrapped assets; no completed audit report
Confirmation policy	Provisional recommendation only	Requires client/exchange approval
Exchange transfer opening	Business date/time pending	Do not equate technical availability with listing opening

2. Explorer gate — client explorer team / Adnan Dev 2

- ☒ `https://lithoscan.ai` returns the production mainnet UI without redirect, challenge, or HTTP 403/5xx for normal unauthenticated users.
- ☒ Explorer identifies Cosmos chain `lithosphere_9005-1` and EVM chain `9005`.
- ☐ Latest height and block time match two independent node endpoints.
- ☐ Height 1 matches hash `7418C1962B64597EE91D6747ECE3D5325C8B17B261E4C0E4A109A9BAFE74F509`.
- ☐ Search works for block height/hash, transaction hash, `0x` address, and `litho1` address.
- ☐ Native balances display 18 decimals without floating-point loss.
- ☐ Successful and failed EVM receipts are distinguishable.

- ☐ LEP100 pages identify assets by contract address and decode `Transfer` logs correctly.
- ☐ Explorer/API does not display testnet LEP100 addresses as mainnet assets.
- ☒ TLS and the restricted rollback procedure passed cutover validation; cache, rate-limit, and error-page policy remain routine operational checks.
- ☒ Public smoke-test evidence and explicit cutover approval are archived in the cutover record and `lithoscan-window-close.json`.

3. Release-artifact gate — infrastructure/release owner

- ☐ Audit public Lithosphere resource/docs pages and remove any stale designation of chain IDs `700777` or `900523` as mainnet; the only exchange mainnet identity in this package is `lithosphere_9005-1` / EVM `9005`.
- ☐ Publish a public or exchange-accessible immutable source tag/commit.
- ☐ Publish the Linux x86_64 mainnet binary at a stable HTTPS release URL.
- ☐ Publish SHA-256 `0546677a9cf3a7f458797b65181a46f21c89185933e832d89ce728a144fd258c`.
- ☐ Publish artifact signature and signer-verification instructions.
- ☐ Include fixed-supply patch, build script, dependency/SBOM artifacts, and focused-test results in the release.
- ☐ Confirm the public genesis URL body and response header match SHA-256 `13e4875b4a9dddc63bdfbd4968c7265f9bbc49218b59c5b49231a56fa313046f`.

4. Snapshot decision — node operations owner

Select and document one:

- ☐ No public snapshot at listing; exchange syncs from genesis and peers.
- ☐ Publish a snapshot with chain ID, height, block hash, creation time, format/compression, size, SHA-256, binary compatibility, and restore test.

Never publish a private validator data directory, consensus key, signing state, or testnet snapshot.

5. Asset and LEP100 gate — protocol/token owner

- ☐ Publish the signed machine-readable mainnet LEP100 registry.
- ☐ For every asset, include chain ID, contract, deployment transaction, name, symbol, decimals, total supply, source release, and runtime-code hash.
- ☐ Complete reproducible bytecode verification for every listed contract.
- ☐ Record issuer and ownership/control addresses.
- ☐ Confirm no testnet contract is included.
- ☐ State whether each contract is the unmodified LEP100 reference or a separately audited modification.
- ☐ Provide token-specific audit reports required by the exchange.

6. Wallet gate — wallet/product owner

- ☐ Confirm Thanos Wallet supports EVM chain `9005`, the production RPC, and the approved explorer URL.
- ☐ Test new wallet, restore, native send/receive, LEP100 send/receive, fee display, wrong-network rejection, and transaction-history links.
- ☐ Publish direct official web/mobile/extension links and version numbers.
- ☐ Confirm whether MetaMask is an officially supported fallback or only a compatible third-party wallet.

7. Exchange policy gate — accountable client and exchange

- ☐ Approve normal native deposit confirmations.
- ☐ Approve high-value native deposit confirmations.
- ☐ Approve LEP100 deposit confirmations.
- ☐ Approve minimum deposits and withdrawals, fees, and dust policy.
- ☐ Approve one-validator operational risk or close the validator-set gate.
- ☐ Approve deposit/withdrawal pause triggers.
- ☐ Approve final mainnet transfer-opening date and UTC timestamp.
- ☐ Approve the MEXC/community warning text.

Suggested provisional values in the draft are 20 blocks normally and 100 blocks for high value. They are not official until signed here.

8. Bridge statement

Select one:

- ☐ Listing is native mainnet LITHO/LEP100 only; bridge and wrapped assets are explicitly out of scope and disabled.
- ☐ A separate bridge integration is requested; provide deployed addresses, routes, operational controls, and the completed independent audit before enabling it.

An audit RFQ or internal test report is not a completed bridge audit.

9. Final validation

- ☐ Run every API example against the production endpoint and an exchange node; retain sanitized output.
- ☐ Run native deposit and withdrawal end to end.
- ☐ Run at least one approved LEP100 deposit, sweep, and withdrawal end to end.
- ☐ Test failed/reverted transactions and confirm no credit.
- ☐ Test duplicate replay and confirm exactly-once credit.
- ☐ Test node restart, indexer replay, RPC outage, peer loss, and alerting.
- ☐ Confirm no secrets, private keys, credentials, or private contact details are present in the external package.
- ☐ Legal/compliance and accountable technical owners approve release.

10. Google Docs publication

After all required gates are complete:

1. Regenerate the combined PDF and DOCX from the reviewed Markdown source.
2. Create a restricted Google Doc titled `LITHO Mainnet Exchange Integration — v1.0 — YYYY-MM-DD`.
3. Import the DOCX, verify tables/code formatting and every link, and make the first line state the approved version and UTC review date.
4. Give edit access only to named client owners; give the exchange review team view/comment access as agreed.
5. Attach or link immutable checksums for genesis, binary, PDF, and source tag.
6. Record the Google Doc URL and access owner below.

Field	Value
Google Doc URL	`TBD`
Document owner	`TBD`
Final version/date	`TBD`
Client approver	`TBD`
Exchange recipient	`TBD`

Controlled source: EXCHANGE_DOCUMENTATION_RELEASE_CHECKLIST.md